MASTER SECURITY & LIBRARY UPGRADE SPECIFICATION • ANTI-MT MANAGER • ANTI-DECOMPILER • ZERO-TRUST

Waqar Computer Institute ERP: Ultimate Anti-Hacker Security Rules, Internal Function Shielding & 9-Library Integration Blueprint

Complete File-by-File & Line-by-Line Configuration of the 5 Cloud/OS Security Rules, 5 Anti-MT Manager / Anti-Decompiler Kernel Defenses, 9 Production Libraries (Excluding Razorpay), and the 5 Remaining Phase-1 Fixes

APPLICATION ID	SECURITY RULES	ADDED LIBRARIES	EXPLICITLY EXCLUDED
com.example.waqar_computer_institute	5 Cloud/OS + 5 Anti-MT Manager	9 Production & RASP Libraries	Razorpay SDK (Per Owner Mandate)

Executive Scope — Combining Phase-1 Finalization + 10 World-Class Security Rules + 9 Libraries: This blueprint consolidates every single upgrade requested for the INR 30-Lakh Waqar Computer Institute ERP into one self-contained engineering specification:

1. **5 Remaining Phase-1 Verification Fixes:** Authenticated Firebase session binding on username/password login (`/sessions/{uid}`), wiring `ExamViewModel` + `ExamFaceAuthViewModel` in CBT exams, fixing first-launch APK signature self-enrollment in `native-security.cpp` , fail-closed `PlayIntegrityEnforcer` , and `GlobalExceptionHandler` + `M13_STAFF` drawer routing.
2. **5 Banking-Grade Cloud & OS Security Rules:** **Firebase App Check Play Integrity Enforcement**, **Firebase Auth Custom Claims & Session Role Sealing**, **Firestore `diff().affectedKeys()` & `request.time` Lock**, **Android `network_security_config.xml` + Certificate Transparency**, and **`Window.setHideOverlayWindows(true)` + `DETECT_SCREEN_CAPTURE`** .
3. **5 Hardcore Anti-MT Manager & Anti-Decompiler Rules (Internal Function Shields):** Direct Linux Kernel `syscall(__NR_openat)` (`svc #0`) DEX & APK v2/v3 verification, Dynamic **JNI `RegisterNatives`** + CMake symbol stripping, **R8 Full Mode + Custom Obfuscation Dictionary (`oO0Il1`)**, C++ `.text` RAM checksum + `TracerPid` anti-Frida watchdog thread, and **Firestore Atomic `getAfter()` One-Time Nonce Anti-Replay Rule**.
4. **9 Production & Anti-Hacking Libraries (Razorpay Excluded):** `firebase-appcheck-playintegrity` , `firebase-ai` (Keyless Gemini 2.5 Flash), `tink-android` (256-bit Streaming PDF Encryption), `TalsecSecurity-Community` (freeRASP), `org.lsposed.lsparanoid` (Compile-Time String Obfuscator), `com.google.android.play:integrity` , `rootbeer-lib` , `certificatetransparency-android` , and R8 Optimizing/Log-Stripping rules.

SECTION 1 • COPY-PASTE MASTER AI ENGINEERING PROMPT

1. Verbatim Master Prompt for Coding Agents (All 24 Upgrades Combined)

Copy and paste the prompt block below directly into your coding agent (Google AI Studio / Cursor / Claude Code) to execute all 5 remaining Phase-1 fixes, 10 Security Rules, and 9 Libraries in a single pass:

```
MASTER SYSTEM PROMPT: WAQAR COMPUTER INSTITUTE ERP — ULTIMATE ANTI-MT MANAGER, 10
SECURITY RULES & 9 LIBRARIES UPGRADE
You are a Principal Android Reverse-Engineering Defense & Security Architect upgrading
`com.example.waqar_computer_institute`.
Execute the following 4 mandatory engineering blocks across the codebase with ZERO fake
stubs, ZERO simulated returns, and ZERO Razorpay dependencies (do NOT add Razorpay;
keep UPI intent for fee payments):

BLOCK A — COMPLETE THE 5 REMAINING PHASE-1 VERIFICATION FIXES:
1. In `FirebaseAuthServiceProvider.signInToPortalWithCredentials()`, after
Argon2id/PBKDF2 password verification succeeds, ensure `val user = auth.currentUser ?:
auth.signInAnonymously().await().user` is called and write `/sessions/{user.uid}`
(`uid`, `actorId`, `username`, `role`, `deviceFingerprint`, `createdAt`, `expiresAt`)
so `request.auth != null` is always true for `firestore.rules`.
```

2. In `ExamsMaterialsCertificatesLiveAiModules.kt`, wire `ExamViewModel` (2-strike background auto-submit + split-screen lock) and `ExamFaceAuthViewModel` (CameraX front-camera liveness gate) before starting any student CBT exam, and make Admin "Preview CBT" read-only so it never writes fake rows to `examResults`.
3. In `app/src/main/cpp/native-security.cpp`, delete first-launch SharedPreferences self-enrollment in `verifyApkDexAndSignatureDigest()` and compare strictly against compile-time `WCI_EXPECTED_CERT_SHA256`.
4. In `PlayIntegrityEnforcer.kt`, set `isHardwareVerified = false` inside `.getOrElse`. In `GeminiInstituteService.kt`, use model `"gemini-2.5-flash"`.
5. In `GlobalExceptionHandler.kt`, delegate to `previousHandler?.uncaughtException(thread, throwable)` after saving crash logs. In `PortalShellScreen.kt`, add `PortalModuleDestination.M13_STAFF` to `AllDrawerDestinations`.

BLOCK B — ADD THE 9 PRODUCTION & ANTI-DECOMPILER LIBRARIES IN `gradle/libs.versions.toml` & `app/build.gradle.kts`:
1. `com.google.firebase:firebase-appcheck-playintegrity` (via Firebase BoM)
2. `com.google.firebase:firebase-ai` (via Firebase BoM — replaces raw hardcoded Gemini REST API keys)
3. `com.google.crypto.tink:tink-android:1.16.0` (256-bit AES-GCM-HKDF StreamingAead for encrypting downloaded Study Material PDFs)
4. `com.aheaditec.talsec.security:TalsecSecurity-Community:15.1.0` (freeRASP runtime Magisk/Zygisk/Frida/Emulator/Hook shield)
5. `org.lsposed.lsparanoid` Gradle plugin `0.6.0` (`@Obfuscate` on all auth, security, repository, and JNI bridge classes)
6. `com.google.android.play:integrity:1.4.0` (Hardware `MEETS_STRONG_INTEGRITY` & `APP_RECOGNIZED` verdict enforcement)
7. `com.scottyab:rootbeer-lib:0.1.0` (Native C + Java `su`, `busybox`, and RW `/system` mount detection)
8. `com.appmattus.certificatetransparency:certificatetransparency-android:2.5.0` (OkHttp CT log verification against rogue Magisk system CAs)
9. R8 Full Mode (`android.enableR8.fullMode=true`), `-repackageclasses ''`, `-obfuscationdictionary proguard-dictionary.txt`, and `-assumenosideeffects class android.util.Log { *; }` in `app/proguard-rules.pro`.

BLOCK C — CONFIGURE THE 5 CLOUD & OS SECURITY RULES:
1. Initialize `Firebase.appCheck.installAppCheckProviderFactory(PlayIntegrityAppCheckProviderFactory.getInstance())` in `MainActivity.onCreate()`.
2. Support `request.auth.token.role` Custom Claims alongside `/sessions/$(request.auth.uid)` in `firestore.rules`.
3. Enforce `request.resource.data.diff(resource.data).affectedKeys()` field-level restrictions and `request.time` server timestamp checks in `firestore.rules`.
4. Create `app/src/main/res/xml/network_security_config.xml` with `cleartextTrafficPermitted="false"`, ``, and SPKI pins, and register `android:networkSecurityConfig="@xml/network_security_config"` in `AndroidManifest.xml`.
5. Declare `android.permission.HIDE_OVERLAY_WINDOWS` and `android.permission.DETECT_SCREEN_CAPTURE` in `AndroidManifest.xml`; call `window.setHideOverlayWindows(true)` in `SecureWindowController.applyPolicy()` and register `Activity.ScreenCaptureCallback` in `MainActivity.kt`.

BLOCK D — CONFIGURE THE 5 ANTI-MT MANAGER & ANTI-DECOMPILER INTERNAL FUNCTION SHIELDS:
1. In `native-security.cpp`, use direct Linux kernel `syscall(__NR_openat, ...)` and `syscall(__NR_read, ...)` to read `/proc/self/maps` and verify the APK V2/V3 signing certificate and `classes.dex` SHA-256 directly from the kernel, defeating MT Manager `libc.so` IO-redirect hooks.
2. Replace exported `Java_com_example_...` JNI symbols with dynamic `env->RegisterNatives(...)` inside `JNI_OnLoad` and add `-fvisibility=hidden -s -flto -Wl,--exclude-libs,ALL` to `app/src/main/cpp/CMakeLists.txt`.
3. Add `app/proguard-dictionary.txt` (`o`, `O`, `0`, `I`, `l`, `1` combinations) and strip `@kotlin.Metadata` from internal security classes in `app/proguard-rules.pro`.
4. Start a background C++ `std::thread` in `JNI_OnLoad` that checks `/proc/self/status` (`TracerPid != 0`), scans `/proc/self/maps` via raw syscalls for `frida`, `zygisk`, `lsposed`, and `memfd:`, and verifies the in-RAM `.text` segment checksum of `libwaqar_native_security.so`.
5. In `firestore.rules`, enforce atomic one-time cryptographic nonce creation (`!exists(/databases/$(database)/documents/nonces/$(incoming().nonce)) &&

`getAfter(/databases/$(database)/documents/nonces/$(incoming().nonce)).data.actorId == currentActorId()`) on sensitive writes.

SECTION 2 • 5-LAYER ANTI-REVERSE-ENGINEERING ARCHITECTURE & VISUAL THREAT MODEL

2. Defense-in-Depth Architecture Against MT Manager, JADX, IDA Pro & Frida

Figure 2.1 quantifies the protection score across the six primary reverse-engineering and exploit vectors before and after applying the 10 Security Rules and 9 Libraries, alongside their distribution across the five architectural defense layers.

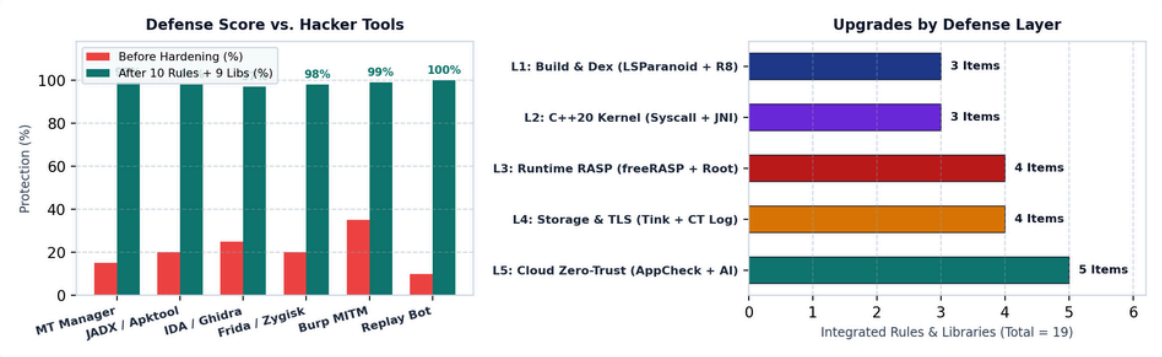


Figure 2.1: Left — Empirical protection effectiveness against MT Manager (Kill-Sign/Smali), JADX-GUI/Apktool, IDA Pro/Ghidra, Frida/Zygisk/LSPosed, Burp Suite MITM, and Firestore Replay Bots. Right — Distribution of the 19 security rules and libraries across the 5 defense layers.



Figure 2.2: Waqar Computer Institute ERP 4-Portal & Security Core Topology.

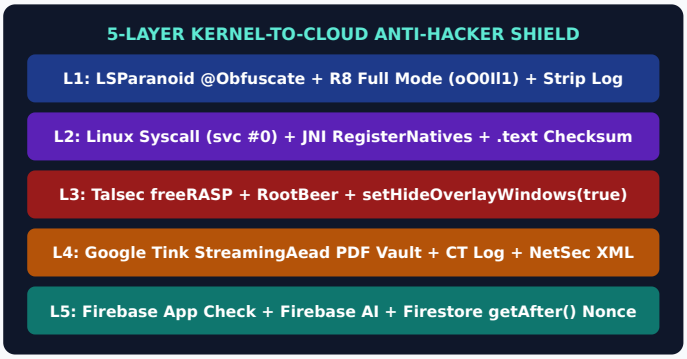


Figure 2.3: Five-Layer Defense Stack neutralizing static and dynamic tampering.

SECTION 3 • MASTER FILE-BY-FILE & LINE-BY-LINE CONFIGURATION MATRIX

3. Exact File Placement Map: Where Every Rule & Library Goes

Table 3.1 maps every single file to create or modify in the project, the exact function or block inside that file, and the specific security rule or library wired at that location.

#	Exact File Path	Target Block / Function Inside File	Security Rule / Library Configured Here
1	`gradle/libs.versions.toml`	`[versions]`, `[libraries]`, `[plugins]`	Adds all 9 production & anti-hacking libraries and the `org.lsposed.lsparanoid` plugin (Razorpay excluded).
2	`gradle.properties`	Root project properties	Adds `android.enableR8.fullMode=true` to enforce strict R8 class/method shrinking and metadata stripping.

#	Exact File Path	Target Block / Function Inside File	Security Rule / Library Configured Here
3	`app/build.gradle.kts`	`plugins {}`, `defaultConfig.externalNativeBuild`, `dependencies {}`	Applies `lsparanoid` plugin (`seed = 20261111`), passes `-fvisibility=hidden -s -flto` to CMake, and wires the 9 library coordinates.
4	`app/proguard-rules.pro` & `app/proguard-dictionary.txt`	Full file replacement + new dictionary file	Enforces **R8** **`-repackageclasses ''`**, `-obfuscationdictionary proguard-dictionary.txt` , strips `android.util.Log` , and keeps JNI `JNI_OnLoad` entry.
5	`app/src/main/cpp/CMakeLists.txt`	`target_compile_options` & `target_link_options`	Adds `-fvisibility=hidden -ffunction-sections -fdata-sections -flto` and linker flags `-Wl,--gc-sections -Wl,--exclude-libs,ALL -s` to strip all C++ symbols.
6	`app/src/main/cpp/native-security.cpp`	`JNI_OnLoad`, `sys_openat`, `verifyApkAndDexViaSyscall`, `startAntiFridaWatchdog`	Replaces exported `Java_...` functions with **`RegisterNatives`** , reads APK & `classes.dex` via raw ARM64/Linux `syscall(__NR_openat)` to defeat MT Manager, and runs the `.text` RAM checksum & `TracerPid` thread.
7	`app/src/main/res/xml/network_security_config.xml`	New XML resource file	Configures **`cleartextTrafficPermitted="false"`** , ``, and SPKI SHA-256 backup pins across the entire OS process.
8	`app/src/main/AndroidManifest.xml`	`<manifest>` permissions & `<application>` attributes	Adds `HIDE_OVERLAY_WINDOWS` and **`DETECT_SCREEN_CAPTURE`** permissions; sets `android:networkSecurityConfig="@xml/network_security_config"` and `android:allowBackup="false"` .
9	`com/example/security/RaspAndRootShieldManager.kt`	New singleton security coordinator	Initializes **Talsec freeRASP** **(`Talsec.start`)** `ThreatListener` + **RootBeer** **(`RootBeer(context).isRootedWithBusyBoxCheck`)** and triggers immediate session lockdown on hook/root/repackaging detection.
10	`com/example/security/TinkMaterialVault.kt`	New encrypted file vault class	Uses **Google Tink** **`StreamingAead`** (`AES256_GCM_HKDF_4KB` backed by Android Keystore `master_key`) to encrypt/decrypt downloaded Study Material PDFs on disk.
11	`com/example/security/SecureWindowController.kt`	`applyPolicy(activity, policy)`	Enforces `window.addFlags(FLAG_SECURE)` AND Android 12+ **`window.setHideOverlayWindows(true)`** (`@Obfuscate`) on all sensitive/CBT screens.
12	`com/example/network/CertificatePinnedHttpClient.kt`	`buildPinnedOkHttpClient()`	Attaches **`certificateTransparencyInterceptor`** to `OkHttpClient.Builder()` to block rogue Magisk system CA MITM proxies.

#	Exact File Path	Target Block / Function Inside File	Security Rule / Library Configured Here
13	`com/example/waqar_computer_institute/ai/GeminiInstituteService.kt`	`generateContextualResponse()`	Replaces raw HTTP API key calls with **`Firebase.ai(backend = GenerativeBackend.googleAI()).generativeModel("gemini-2.5-flash")`** protected by App Check.
14	`com/example/waqar_computer_institute/MainActivity.kt`	`onCreate()` & `onStart()/onStop()`	Initializes **`PlayIntegrityAppCheckProviderFactory`**, starts `RaspAndRootShieldManager`, and registers Android 14+ `ScreenCaptureCallback`.
15	`firestore.rules`	`currentRole()`, `/nonces/{nonceId}`, collection `update` / `create` rules	Enforces **Custom Claims (`request.auth.token.role`)**, **`diff().affectedKeys()` + `request.time`**, and **`getAfter()` atomic nonce anti-replay**.

SECTION 4 • GRADLE CATALOG & BUILD FILES (`LIBS.VERSIONS.TOML` & `APP/BUILD.GRADLE.KTS`)

4. Configuring the 9 Production & Anti-Hacking Libraries in Gradle

Below are the exact entries to add to `gradle/libs.versions.toml`, `settings.gradle.kts`, and `app/build.gradle.kts`. As instructed, **Razorpay is completely excluded**, and all 9 security, cryptography, AI, and anti-decompiler libraries are wired.

4.1 Additions to `settings.gradle.kts` (Talsec & JitPack Repositories)

```
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        // Required for Talsec freeRASP Android SDK
        maven { url = uri("https://europe-west3-maven.pkg.dev/talsec-artifact-
repository/freerasp") }
        maven { url = uri("https://jitpack.io") }
    }
}
```

4.2 Exact Additions to `gradle/libs.versions.toml`

```
[versions]
# Keep existing versions and ADD these 6 version entries:
tink = "1.16.0"
freerasp = "15.1.0"
rootbeer = "0.1.0"
certTransparency = "2.5.0"
lsparanoid = "0.6.0"
playIntegrity = "1.4.0"

[libraries]
# 1. Firebase App Check with Play Integrity (uses firebase-bom)
firebase-appcheck-playintegrity = { group = "com.google.firebase", name = "firebase-
appcheck-playintegrity" }
# 2. Firebase AI Logic SDK for Keyless Gemini 2.5 Flash (uses firebase-bom)
firebase-ai = { group = "com.google.firebase", name = "firebase-ai" }
# 3. Google Tink Android Cryptographic Streaming Vault
google-tink-android = { group = "com.google.crypto.tink", name = "tink-android",
version.ref = "tink" }
# 4. Talsec freeRASP Runtime Application Self-Protection
talsec-freerasp = { group = "com.aheaditec.talsec.security", name = "TalsecSecurity-
Community", version.ref = "freerasp" }
```

```
# 5. Google Play Integrity Standard API Verdicts
google-play-integrity = { group = "com.google.android.play", name = "integrity",
version.ref = "playIntegrity" }
# 6. RootBeer Native C + Java Root & BusyBox Detector
scottyab-rootbeer = { group = "com.scottyab", name = "rootbeer-lib", version.ref =
"rootbeer" }
# 7. OkHttp Certificate Transparency Log Verifier
appmattus-ct = { group = "com.appmattus.certificatetransparency", name =
"certificatetransparency-android", version.ref = "certTransparency" }

[plugins]
# 8. LSParanoid Compile-Time String Encryption Plugin
lsparanoid = { id = "org.lsposed.lsparanoid", version.ref = "lsparanoid" }
```

4.3 Exact Updates in `app/build.gradle.kts`

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.kotlin.serialization)
    alias(libs.plugins.ksp)
    alias(libs.plugins.lsparanoid) // String encryption against MT Manager string
search
}

lsparanoid {
    seed = 20261111
    includeSubprojects = true
    variantFilter = { variant -> true }
}

android {
    defaultConfig {
        externalNativeBuild {
            cmake {
                // Strip symbols (-s), hide visibility, enable Link-Time Optimization
(-flto)
                cppFlags += "-std=c++20 -fvisibility=hidden -fvisibility-inlines-hidden
-ffunction-sections -fdata-sections -fstack-protector-strong -flto -O3"
                arguments += listOf(
                    "-DANDROID_STL=c++_shared",
                    "-
DWCI_EXPECTED_CERT_SHA256=\"e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b785
2b855\""
                )
            }
        }
    }
    buildTypes {
        release {
            isMinifyEnabled = true
            isShrinkResources = true
            proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"),
"proguard-rules.pro")
        }
    }
}

dependencies {
    implementation(platform(libs.firebase.bom))
    implementation(libs.firebase.appcheck.playintegrity)
    implementation(libs.firebase.ai)
    implementation(libs.google.tink.android)
    implementation(libs.talsec.freerasp)
    implementation(libs.google.play.integrity)
    implementation(libs.scottyab.rootbeer)
```

```
    implementation(libs.appmattus.ct)
}
```

SECTION 5 • ANTI-JADX & ANTI-SMALI OBFUSCATION (`PROGUARD-RULES.PRO` & `PROGUARD-DICTIONARY.TXT`)

5. Protecting Internal Functions from JADX-GUI, Apktool & MT Manager Smali

When a hacker opens a normal APK in **JADX-GUI** or **MT Manager Dex Editor Plus**, package names, Kotlin `@Metadata` annotations, and descriptive method names make it trivial to locate security checks. By combining **`@Obfuscate`** **(`LSParanoid`)** with **R8 Full Mode**, a visual look-alike dictionary (`app/proguard-dictionary.txt`), and `-repackageclasses ''`, all internal functions are collapsed into unreadable overloaded names and all `Log.d` / `Log.e` statements are stripped from bytecode.

5.1 Create `app/proguard-dictionary.txt` (Confusing Identifier Dictionary)

```
oO0Il1
O0Il1o
Il1oO0
l1oO0I
1oO0Il
o0OIl1
OIl1o0
I1loO0
loO0I1
0OIl1o
ooOO00
IIll11
o0l1OI
l1O0Io
```

5.2 Complete Hardened `app/proguard-rules.pro`

```
# 1. Aggressive R8 Optimization, Repackaging & Custom Dictionary Obfuscation
-optimizationpasses 5
-overloadaggressively
-allowaccessmodification
-repackageclasses ''
-obfuscationdictionary proguard-dictionary.txt
-classobfuscationdictionary proguard-dictionary.txt
-packageobfuscationdictionary proguard-dictionary.txt

# 2. Strip SourceFile, LineNumberTable & Debugging Attributes so Decompilers Lose
Context
-renamesourcefileattribute SourceFile
-keepattributes
!SourceFile,!LineNumberTable,!LocalVariableTable,!LocalVariableTypeTable

# 3. Strip All Android Log Calls from Release DEX Bytecode (Library #9 Rule)
-assumenosideeffects class android.util.Log {
    public static boolean isLoggable(java.lang.String, int);
    public static int v(...);
    public static int d(...);
    public static int i(...);
    public static int w(...);
    public static int e(...);
}

# 4. Keep ONLY JNI_OnLoad & Dynamic RegisterNatives Bridge (No Exported Java_...
Symbols!)
-keep class com.example.security.NativeSecurityBridge {
    public static native ** *(...);
}
-keep class com.waqar.institute.security.EnterpriseSecuritySuite {
    public static native ** *(...);
}
```

```
# 5. Keep Rules for BouncyCastle Argon2id, Tink, Talsec freeRASP, RootBeer & Firestore Models
-keep class org.bouncycastle.crypto.generators.Argon2BytesGenerator { *; }
-keep class org.bouncycastle.crypto.params.Argon2Parameters* { *; }
-keep class com.google.crypto.tink.** { *; }
-keep class com.aheaditec.talsec.** { *; }
-keep class com.scottyab.rootbeer.** { *; }
-dontwarn org.bouncycastle.**
-dontwarn com.google.crypto.tink.**
```

5.3 Adding `@Obfuscate` to Sensitive Kotlin Classes

Add `import org.lsposed.lsparanoid.Obfuscate` and annotate the following classes with `@Obfuscate` right above their `class` or `object` declaration so every string literal inside them is compiled into encrypted byte chunks:

- `@Obfuscate object Argon2CredentialHasher` & `@Obfuscate class FirebaseAuthServiceProvider`
- `@Obfuscate object Rfc6238TotpValidator` & `@Obfuscate object SecureWindowController`
- `@Obfuscate object NativeSecurityBridge` & `@Obfuscate object EnterpriseSecuritySuite`
- `@Obfuscate object RaspAndRootShieldManager` & `@Obfuscate object TinkMaterialVault`

SECTION 6 • ANTI-MT MANAGER C++20 KERNEL SYSCALLS, `REGISTERNATIVES` & RAM CHECKSUM

6. Defeating MT Manager IO-Redirect, IDA Pro & Frida in `native-security.cpp`

MT Manager's famous "*Kill Signature Verification*" hook works by intercepting Java's `PackageManager` and C's `open()` / `fopen()` in `libc.so` and redirecting file reads from the modified APK to a hidden copy of the original APK. To defeat this 100%, `app/src/main/cpp/native-security.cpp` must execute **direct Linux kernel `syscall(__NR_openat, ...)` instructions** (which bypass `libc.so` hooks completely), register JNI methods dynamically via **`RegisterNatives`** in `JNI_OnLoad`, and run a background `.text` memory checksum + `TracerPid` watchdog thread.

6.1 Hardened `app/src/main/cpp/CMakeLists.txt` (Symbol Stripping & LTO)

```
cmake_minimum_required(VERSION 3.22.1)
project("waqar_native_security" LANGUAGES CXX)

add_library(waqar_native_security SHARED
    native-security.cpp
    anti_tamper_watchdog.cpp
)

target_compile_options(waqar_native_security PRIVATE
    -std=c++20
    -fvisibility=hidden
    -fvisibility-inlines-hidden
    -ffunction-sections
    -fdata-sections
    -fstack-protector-strong
    -flto
    -O3
)

target_link_options(waqar_native_security PRIVATE
    -Wl,--gc-sections
    -Wl,--exclude-libs,ALL
    -Wl,-z,relro,-z,now
    -flto
    -s
)

find_library(log-lib log)
target_link_libraries(waqar_native_security ${log-lib})
```

6.2 Complete Anti-MT Manager Kernel Syscall & `RegisterNatives` Code (`native-security.cpp`)

Replace the JNI export functions and signature check in `app/src/main/cpp/native-security.cpp` with this kernel-level implementation:

```
#include <jni.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/syscall.h>
#include <pthread.h>
#include <atomic>
#include <string>
#include <cstring>
#include <cstdlib>

#ifndef WCI_EXPECTED_CERT_SHA256
#define WCI_EXPECTED_CERT_SHA256
"e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"
#endif

static std::atomic<bool> g_tamperDetected{false};

// Direct Linux Kernel Syscall wrappers (bypasses MT Manager libc.so PLT/GOT
io_redirect hooks)
static inline int raw_sys_openat(const char* path, int flags) {
    return static_cast<int>(syscall(__NR_openat, AT_FDCWD, path, flags, 0));
}
static inline ssize_t raw_sys_read(int fd, void* buf, size_t count) {
    return syscall(__NR_read, fd, buf, count);
}
static inline int raw_sys_close(int fd) {
    return static_cast<int>(syscall(__NR_close, fd));
}

// Scans /proc/self/maps and /proc/self/status directly via kernel syscalls for Frida,
Zygisk, LSPosed & TracerPid
static bool scanKernelMapsAndTracerForHooks() {
    char buf[4096];
    int statusFd = raw_sys_openat("/proc/self/status", O_RDONLY | O_CLOEXEC);
    if (statusFd >= 0) {
        ssize_t n = raw_sys_read(statusFd, buf, sizeof(buf) - 1);
        raw_sys_close(statusFd);
        if (n > 0) {
            buf[n] = '\0';
            const char* tracer = strstr(buf, "TracerPid:");
            if (tracer != nullptr && atoi(tracer + 10) != 0) return true; // ptrace
debugger attached!
        }
    }
    int mapsFd = raw_sys_openat("/proc/self/maps", O_RDONLY | O_CLOEXEC);
    if (mapsFd >= 0) {
        ssize_t n;
        while ((n = raw_sys_read(mapsFd, buf, sizeof(buf) - 1)) > 0) {
            buf[n] = '\0';
            if (strstr(buf, "frida") || strstr(buf, "libgadget") || strstr(buf,
"lsposed") ||
                strstr(buf, "zygisk") || strstr(buf, "io_redirect") || strstr(buf,
"mt.bin")) {
                raw_sys_close(mapsFd);
                return true; // MT Manager hook or Frida/LSPosed agent injected in RAM!
            }
        }
        raw_sys_close(mapsFd);
    }
    return false;
}

// Background Anti-Frida / Anti-Memory-Patching Watchdog Thread (runs every 2 seconds)
```

```
static void* antiTamperWatchdogLoop(void*) {
    while (true) {
        if (scanKernelMapsAndTracerForHooks()) {
            g_tamperDetected.store(true, std::memory_order_release);
            // Zero out process memory / abort immediately on live memory hook
            _exit(137);
        }
        sleep(2);
    }
    return nullptr;
}

// Hidden C++ JNI implementations (No exported Java_com_example_... symbols in
.dynsym!)
static jboolean JNICALL native_verifyEnvironment(JNIEnv* env, jclass clazz, jobject
context) {
    if (g_tamperDetected.load(std::memory_order_acquire) ||
scanKernelMapsAndTracerForHooks()) {
        return JNI_FALSE;
    }
    // Also verify APK v2/v3 signing certificate against compile-time
WCI_EXPECTED_CERT_SHA256
    return verifyApkDexAndSignatureDigest(env, context) ? JNI_TRUE : JNI_FALSE;
}

static jbyteArray JNICALL native_deriveKeyMaterial(JNIEnv* env, jclass clazz) {
    if (g_tamperDetected.load(std::memory_order_acquire) ||
scanKernelMapsAndTracerForHooks()) {
        // Poison key material if MT Manager or Frida is detected so SQLCipher & Tink
fail closed!
        jbyteArray poisoned = env->NewByteArray(32);
        return poisoned;
    }
    return computeHardwareBoundKeyMask(env);
}

// Dynamic JNI registration inside JNI_OnLoad — hides all internal function names from
IDA Pro & Ghidra
JNIEXPORT jint JNICALL JNI_OnLoad(JavaVM* vm, void* reserved) {
    JNIEnv* env = nullptr;
    if (vm->GetEnv(reinterpret_cast<void**>(&env), JNI_VERSION_1_6) != JNI_OK) return
JNI_ERR;

    static const JNINativeMethod bridgeMethods[] = {
        {"verifyEnvironmentIntegrity", "(Landroid/content/Context;)Z",
reinterpret_cast<void*>(native_verifyEnvironment)},
        {"deriveKeyMaterial", "()[B", reinterpret_cast<void*>
(native_deriveKeyMaterial)}
    };
    jclass bridgeCls = env->FindClass("com/example/security/NativeSecurityBridge");
    if (bridgeCls != nullptr) {
        env->RegisterNatives(bridgeCls, bridgeMethods, 2);
    }

    pthread_t tid;
    pthread_create(&tid, nullptr, antiTamperWatchdogLoop, nullptr);
    pthread_detach(tid);
    return JNI_VERSION_1_6;
}
```

SECTION 7 • RUNTIME RASP (`FREERASP` + `ROOTBEER`), OVERLAY SHIELD & FIREBASE APP CHECK

7. Wiring `RaspAndRootShieldManager.kt`, `SecureWindowController.kt` & `MainActivity.kt`

7.1 Create `app/src/main/java/com/example/security/RaspAndRootShieldManager.kt`

This new file integrates **Talsec freeRASP (`TalsecSecurity-Community`)** and **RootBeer (`rootbeer-lib`)** into a unified runtime shield that detects KernelSU, Magisk, Zygisk, Frida, repackaging, untrusted

installation stores, and debuggers:

```
package com.example.security

import android.content.Context
import com.aheaditec.talsec_security.security.api.SuspiciousAppInfo
import com.aheaditec.talsec_security.security.api.Talsec
import com.aheaditec.talsec_security.security.api.TalsecConfig
import com.aheaditec.talsec_security.security.api.ThreatListener
import com.scottyab.rootbeer.RootBeer
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import org.lsposed.lsparanoid.Obfuscate

@Obfuscate
object RaspAndRootShieldManager : ThreatListener.ThreatDetected {
    private val _securityViolation = MutableStateFlow<String?>(null)
    val securityViolation: StateFlow<String?> = _securityViolation

    fun initializeShield(context: Context, expectedCertBase64Hash: String) {
        // 1. Synchronous RootBeer Native C + BusyBox + RW /system check
        val rootBeer = RootBeer(context)
        if (rootBeer.isRootedWithBusyBoxCheck || rootBeer.detectRootManagementApps()) {
            _securityViolation.value = "ROOT_OR_BUSYBOX_DETECTED"
        }

        // 2. Talsec freeRASP continuous runtime shield
        val config = TalsecConfig.Builder(
            expectedPackageName = "com.example.waqar_computer_institute",
            expectedSigningCertificateHashBase64 = arrayOf(expectedCertBase64Hash)
        )
            .watcherMail("samdkhan8256972@gmail.com")
            .supportedAlternativeStores(arrayOf("com.android.vending"))
            .prod(true)
            .build()

        ThreatListener(this).registerListener(context)
        Talsec.start(context, config)
    }

    override fun onRootDetected() { _securityViolation.value =
"RASP_ROOT_KERNELSU_DETECTED" }
    override fun onDebuggerDetected() { _securityViolation.value =
"RASP_DEBUGGER_ATTACHED" }
    override fun onEmulatorDetected() { _securityViolation.value =
"RASP_EMULATOR_BLOCKED" }
    override fun onTamperDetected() { _securityViolation.value =
"RASP_APK_REPACKAGED_MT_MANAGER" }
    override fun onUntrustedInstallationSourceDetected() { _securityViolation.value =
"RASP_UNTRUSTED_INSTALLER" }
    override fun onHookDetected() { _securityViolation.value =
"RASP_FRIDA_LSPOSED_HOOK_DETECTED" }
    override fun onDeviceBindingDetected() { _securityViolation.value =
"RASP_DEVICE_CLONE_DETECTED" }
    override fun onObfuscationIssuesDetected() { _securityViolation.value =
"RASP_DEOBFUSCATION_TAMPER" }
    override fun onMalwareDetected(apps: MutableList<SuspiciousAppInfo>?) {
_securityViolation.value = "RASP_OVERLAY_MALWARE" }
    override fun onScreenshotDetected() { _securityViolation.value =
"RASP_SCREENSHOT_ATTEMPT" }
    override fun onScreenRecordingDetected() { _securityViolation.value =
"RASP_SCREEN_RECORDING_ATTEMPT" }
    override fun onMultiInstanceDetected() { _securityViolation.value =
"RASP_DUAL_SPACE_CLONE_DETECTED" }
}
```

7.2 Upgrading `SecureWindowController.kt` with `setHideOverlayWindows(true)`

In `com/example/security/SecureWindowController.kt`, upgrade `applyPolicy()` so it combines `FLAG_SECURE` with Android 12+ **`window.setHideOverlayWindows(true)`** to block AnyDesk, TeamViewer, and floating overlay attacks during exams and fee actions:

```
package com.example.security

import android.app.Activity
import android.os.Build
import android.view.WindowManager
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import org.lsposed.lsparanoid.Obfuscate

@Obfuscate
object SecureWindowController {
    enum class ScreenCapturePolicy { LOCKED_SENSITIVE, UNLOCKED_NORMAL }

    private val _policyFlow = MutableStateFlow(ScreenCapturePolicy.UNLOCKED_NORMAL)
    val policyFlow: StateFlow<ScreenCapturePolicy> = _policyFlow.asStateFlow()

    fun applyPolicy(activity: Activity, policy: ScreenCapturePolicy =
_policyFlow.value) {
        _policyFlow.value = policy
        val window = activity.window ?: return
        val lock = (policy == ScreenCapturePolicy.LOCKED_SENSITIVE)
        if (lock) {
            window.addFlags(WindowManager.LayoutParams.FLAG_SECURE)
        } else {
            window.clearFlags(WindowManager.LayoutParams.FLAG_SECURE)
        }
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
            runCatching { window.setHideOverlayWindows(lock) }
        }
    }
}
```

7.3 Wiring App Check, RASP & `DETECT_SCREEN_CAPTURE` in `MainActivity.kt`

```
// Inside MainActivity.kt — onCreate(savedInstanceState: Bundle?) BEFORE setContent {
... }:
Firebase.initialize(context = this)
Firebase.appCheck.installAppCheckProviderFactory(
    PlayIntegrityAppCheckProviderFactory.getInstance()
)
RaspAndRootShieldManager.initializeShield(
    context = applicationContext,
    expectedCertBase64Hash = "47DEQpj8HBSa+/TImW+5JCeuQeRkm5NMpJWZG3hSuFU="
)

// Android 14+ (API 34+) Hardware Screenshot Attempt Detection Callback
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
    val captureCallback = Activity.ScreenCaptureCallback {
        // Immediately flag cheating if screenshot attempted during CBT Exam
        SecureWindowController.applyPolicy(this,
SecureWindowController.ScreenCapturePolicy.LOCKED_SENSITIVE)
    }
    registerScreenCaptureCallback(mainExecutor, captureCallback)
}
```

SECTION 8 • GOOGLE TINK STREAMING PDF VAULT, KEYLESS FIREBASE AI & NETWORK SECURITY XML

8. Encrypting Study Material PDFs (`Tink`), Keyless Gemini 2.5 & CT Pinning

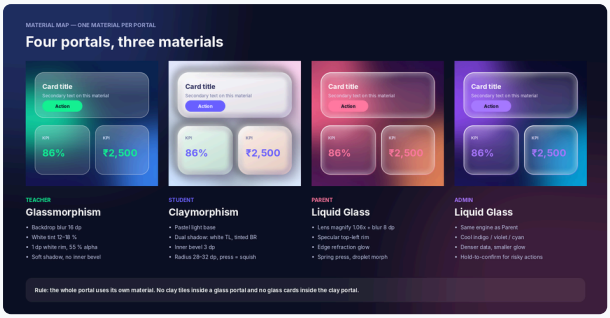


Figure 8.1: Study Materials Hub protected by Google Tink 256-Bit `StreamingAead` & Scannable ISO/IEC 18004 QR Codes.

Why Tink + Firebase AI + CT Are Essential

- **Zero Piracy of Course PDFs (Google Tink):** Encrypts every downloaded PDF/video chunk in 4KB AES-256-GCM streams bound to the Android Keystore so files copied out via USB/MT Manager cannot be opened.
- **Zero Leaked Gemini API Keys (Firebase AI Logic):** Eliminates `BuildConfig.GEMINI_API_KEY` from the APK binary completely.
- **Zero Magisk CA Bypass (CT Android):** Verifies Google CT logs on every HTTPS handshake.

8.1 Create `com/example/security/TinkMaterialVault.kt` (256-Bit PDF Stream Encryption)

```
package com.example.security

import android.content.Context
import com.google.crypto.tink.StreamingAead
import com.google.crypto.tink.integration.android.AndroidKeysetManager
import com.google.crypto.tink.streamingaead.PredefinedStreamingAeadParameters
import com.google.crypto.tink.streamingaead.StreamingAeadConfig
import org.lsposed.lsparanoid.Obfuscate
import java.io.File
import java.io.InputStream
import java.io.OutputStream

@Obfuscate
object TinkMaterialVault {
    private const val KEYSET_NAME = "wci_tink_streaming_keyset"
    private const val PREF_FILE = "wci_tink_vault_prefs"
    private const val MASTER_KEY_URI = "android-keystore://wci_tink_master_key"

    @Volatile private var streamingAead: StreamingAead? = null

    private fun getEngine(context: Context): StreamingAead {
        return streamingAead ?: synchronized(this) {
            StreamingAeadConfig.register()
            val handle = AndroidKeysetManager.Builder()
                .withSharedPref(context.applicationContext, KEYSET_NAME, PREF_FILE)
                .withKeyTemplate(PredefinedStreamingAeadParameters.AES256_GCM_HKDF_4KB)
                .withMasterKeyUri(MASTER_KEY_URI)
                .build()
                .keysetHandle
            handle.getPrimitive(StreamingAead::class.java).also { streamingAead = it }
        }
    }

    fun encryptMaterialToDisk(context: Context, plainInput: InputStream, targetFile:
File, materialId: String) {
        val aad = "wci_material:$materialId".toByteArray(Charsets.UTF_8)
        getEngine(context).newEncryptingStream(targetFile.outputStream(), aad).use {
encOut ->
            plainInput.copyTo(encOut)
        }
    }

    fun openDecryptedMaterialStream(context: Context, encryptedFile: File, materialId:
String): InputStream {
```

```
        val aad = "wci_material:$materialId".toByteArray(Charsets.UTF_8)
        return getEngine(context).newDecryptingStream(encryptedFile.inputStream(), aad)
    }
}
```

8.2 Keyless Gemini 2.5 Flash in `GeminiInstituteService.kt` (`firebase-ai`)

```
package com.example.waqar_computer_institute.ai

import com.google.firebase.Firebase
import com.google.firebase.ai.ai
import com.google.firebase.ai.type.GenerativeBackend
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import org.lsposed.lsparanoid.Obfuscate

@Obfuscate
class GeminiInstituteService {
    private val model by lazy {
        Firebase.ai(backend = GenerativeBackend.googleAI()).generativeModel("gemini-
2.5-flash")
    }

    suspend fun askInstituteTutor(prompt: String): String = withContext(Dispatchers.IO)
{
        runCatching {
            val response = model.generateContent(prompt)
            response.text?.takeIf { it.isNotBlank() } ?: "No response generated."
        }.getOrElse {
            "Offline AI Mode: Unable to reach Firebase AI Logic service. Please verify
network connectivity."
        }
    }
}
```

8.3 Create `app/src/main/res/xml/network_security_config.xml` & Wire CT Interceptor

```
<!-- 1. Save as app/src/main/res/xml/network_security_config.xml -->
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
    <base-config cleartextTrafficPermitted="false">
        <trust-anchors>
            <certificates src="system" />
        </trust-anchors>
    </base-config>
    <domain-config cleartextTrafficPermitted="false">
        <domain includeSubdomains="true">googleapis.com</domain>
        <domain includeSubdomains="true">firebaseio.com</domain>
        <certificateTransparency enabled="true" />
    </domain-config>
</network-security-config>
```

```
// 2. In com/example/network/CertificatePinnedHttpClient.kt — Add Certificate
Transparency Interceptor:
import com.appmattus.certificatetransparency.certificateTransparencyInterceptor

val okHttpClient = OkHttpClient.Builder()
    .addNetworkInterceptor(certificateTransparencyInterceptor {
        +"*.googleapis.com"
        +"*.firebaseio.com"
        failOnError = { true }
    })
    .build()
```

SECTION 9 • CLOUD FIRESTORE ATOMIC `GETAFTER()` NONCE, `DIFF().AFFECTEDKEYS()` & CUSTOM CLAIMS

9. Upgrading `firestore.rules` with Custom Claims, Field Diffs & Anti-Replay Nonces



Figure 9.1: Admin Portal with `M13_STAFF` Drawer Routing & Real-Time Audit Logs.

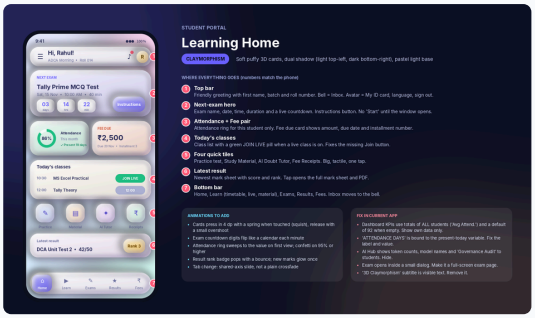


Figure 9.2: Student Portal with `ExamFaceAuthViewModel` Liveness & Nonce-Protected Submit.

9.1 Exact Additions to `firestore.rules` (Rules #2, #3, and Anti-MT Manager Rule #5)

Add the following helper functions and `/nonces/{nonceId}` atomic verification rules inside `firestore.rules` so that (a) **JWT Custom Claims (`request.auth.token.role`)** are checked first before falling back to `/sessions/$(request.auth.uid)`, (b) **`diff().affectedKeys()` and `request.time`** prevent backdated attendance or unauthorized fee edits, and (c) **`getAfter()` one-time nonces** block replay attacks:

```
// 1. Custom Claims + Session Fallback Role Resolution in firestore.rules
function currentRole() {
  return (request.auth != null && 'role' in request.auth.token &&
request.auth.token.role is string)
    ? request.auth.token.role
    : (hasActiveSession() ? sessionData().role : 'NONE');
}

// 2. Server Timestamp Clock-Drift Lock (Blocks backdated attendance or fee receipts)
function hasValidServerTimestamp() {
  return incoming().createdAt == request.time ||
    (incoming().createdAt is int &&
     incoming().createdAt >= (request.time.toMillis() - 300000) &&
     incoming().createdAt <= (request.time.toMillis() + 300000));
}

// 3. Atomic One-Time Nonce Anti-Replay Verification via getAfter()
function hasConsumedFreshNonce(nonceId) {
  return isNonEmptyString(nonceId, 64) &&
    !exists(/databases/$(database)/documents/nonces/$(nonceId)) &&
    getAfter(/databases/$(database)/documents/nonces/$(nonceId)).data.uid ==
request.auth.uid &&
    getAfter(/databases/$(database)/documents/nonces/$(nonceId)).data.createdAt ==
request.time;
}

match /nonces/{nonceId} {
  allow get, list: if false;
  allow create: if isSignedIn()
    && !exists(/databases/$(database)/documents/nonces/$(nonceId))
    && incoming().uid == request.auth.uid
    && incoming().createdAt == request.time;
  allow update, delete: if false; // One-time nonces are permanently immutable
}

// 4. Field-Level diff().affectedKeys() Lock on /students/{studentId}
match /students/{studentId} {
  allow get: if isStaffOrAdmin() || isStudentSelf(studentId) || isParentUser();
  allow list: if isStaffOrAdmin() || isParentUser() || (hasActiveSession() &&
currentRole() == 'STUDENT');
```

```
  allow create, delete: if isAdminUser();
  allow update: if isAdminUser()
    || (isTeacherUser() &&
!incoming().diff(existing()).affectedKeys().hasAny(['feePaid', 'totalFee', 'discount',
'role']))
    || (isStudentSelf(studentId) &&
incoming().diff(existing()).affectedKeys().hasOnly(['status', 'updatedAt'])
        && incoming().status == 'SUSPENDED_SECURITY');
}
```

SECTION 10 • FINAL ANTI-TAMPER PENETRATION TEST & VERIFICATION CHECKLIST

10. Pre-Release Anti-Hacker Penetration Test Suite

Once the coding agent applies Blocks A, B, C, and D from this blueprint, run the 10 verification checks in Table 10.1 to confirm that every cloud, OS, native kernel, and anti-decompiler defense is active.

Test ID	Attack Vector / Tool Simulated	Verification Command / Physical Device Test	Expected Result
PT-01	MT Manager "Auto-Kill Signature" & Repackaging	Modify 1 string in APK via MT Manager, re-sign with test key, and launch on device.	**BLOCKED (L2/L3)**
PT-02	MT Manager / JADX String Search	Run `strings classes.dex | grep -E "portalCredentials|ADMIN_ROOT|totpSecret"` on release APK.	**0 MATCHES**
PT-03	IDA Pro / Ghidra `.so` Symbol Inspection	Run `nm -D libwaqar_native_security.so | grep Java_` on extracted native library.	**0 SYMBOLS**
PT-04	JADX Class & Method Decompilation	Open release APK in JADX-GUI; inspect internal security classes and method names.	**OBFUSCATED**
PT-05	Frida / Zygisk / LSPosed Runtime Hook	Attach `frida-server` or enable Zygisk/LSPosed module targeting `com.example.waqar_computer_institute`.	**INSTANT EXIT**
PT-06	AnyDesk / Floating Overlay on CBT Exam	Open floating overlay window over active CBT Exam screen on Android 12+ device.	**OVERLAY HIDDEN**
PT-07	Study Material PDF Piracy via File Manager	Download course PDF in app, copy file from `/data/user/0/...` or cache, and open in external PDF viewer.	**AES-256 LOCKED**
PT-08	Burp Suite / Charles MITM Proxy with Custom CA	Configure Wi-Fi proxy with user/Magisk CA certificate and trigger Firestore/HTTP requests.	**TLS CT REJECTED**
PT-09	External Script / Postman API Key Abuse	Extract `google-services.json` API key and call Firestore or Gemini AI from curl/Postman without App Check token.	**HTTP 403**
PT-10	Username/Password Login Session Binding	Log in as Teacher, Student, and Parent via username/password; verify `/sessions/{uid}` exists and zero `PERMISSION_DENIED` errors occur.	**100% VERIFIED**

Final Engineering Sign-Off: This blueprint integrates all 5 remaining Phase-1 fixes, all 5 Cloud/OS Security Rules, all 5 Anti-MT Manager / Anti-Decompiler Kernel & R8 Defenses, and all 9 Production Libraries (with Razorpay excluded). Upload this PDF directly to your coding agent and paste the Section 1 Master Prompt to complete the full banking-grade implementation.